

Sorbonne Université
Paradigmes de Programmation Concurrente 51553



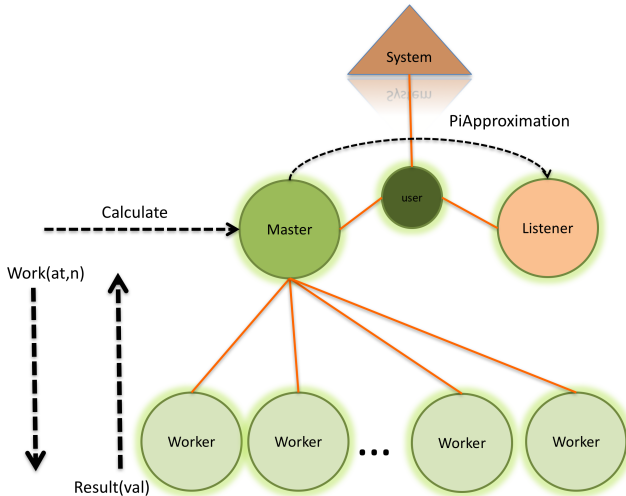
Cours 10 - Le modèle acteur distribué
Akka

Carlos Agon

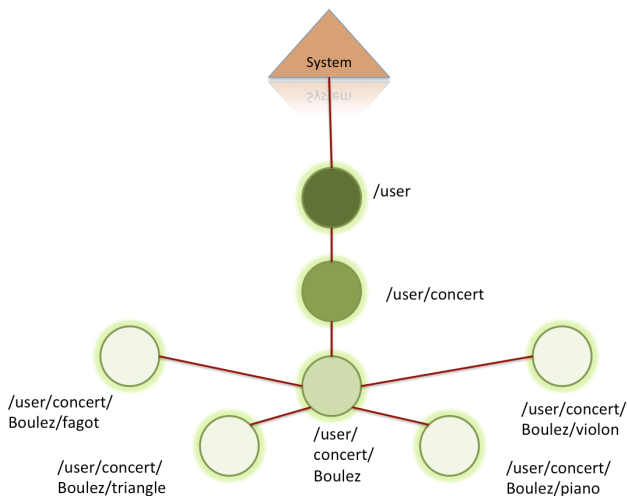
10 décembre 2024

Exemple 4 : pi en distribué

$$\sum_{n=0}^{\infty} \frac{(-1)^n}{2n+1} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots = \frac{\pi}{4}$$



System : Settings e.g. Human-Optimized Config Object Notation (HOCON)



Acteurs distribués

L'ActorSystem n'est pas conscient du fait qu'un sous ensemble de ses enfants puissent être dans des nodes différents.

```
libraryDependencies +=  
  Seq( "com.typesafe.akka" %% "akka-remote" % "2.1.0")
```

A part inclure le module akka-remote, on doit aussi modifier le dossier *application.conf*.

```
akka {  
  actor {  
    provider = "akka.remote.RemoteActorRefProvider"  
    \\ par défaut c'est akka.Local.RemoteActorRefProvider  
  }  
  
  remote {  
    enabled-transports = ["akka.remote.netty.tcp"]  
    netty.tcp {  
      hostname = "129.102.65.14"  
      port = 2552  
    }  
  }  
}
```

Créer et chercher un remote actor

```
val selection =  
    akka.<protocol>://<actor system>@<hostname>:<port>/<actor path>  
  
context.actorSelection("akka.tcp://actorSystemName@10.0.0.1:2552/user/actorName")  
  
selection ! "salut"
```

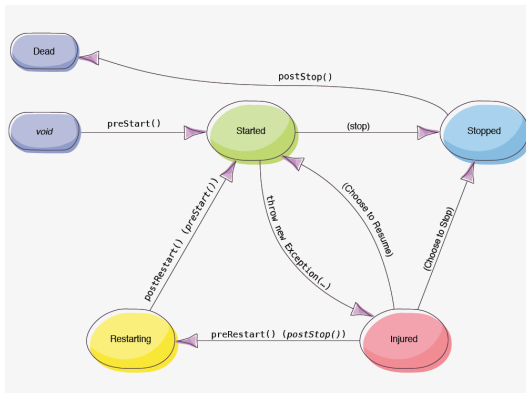
Pour créer des acteurs dans un node remote, on doit modifier *application.conf*.

```
akka {  
  actor {  
    deployment {  
      /sampleActor {  
        remote = "akka.tcp://sampleActorSystem@127.0.0.1:2553"  
      }  
    }  
  }  
}
```

sampleActor est un alias pour sampleActorSystem@127.0.0.1:2553.

```
val myActeur = system.actorOf(Props(sampleActor()), "exemple")
```

Le cycle de vie d'un acteur



- *Creation*, avec `system.actorOf()` ou `context.actorOf()`
- *Stop*, explicite avec `context.stop()` ou `system.stop()`
- *Restart*, on peut recommencer l'acteur suite à une exception.

Exemple 1 : Hook stop et start

Exceptions

Un acteur peut toujours signaler une exception.

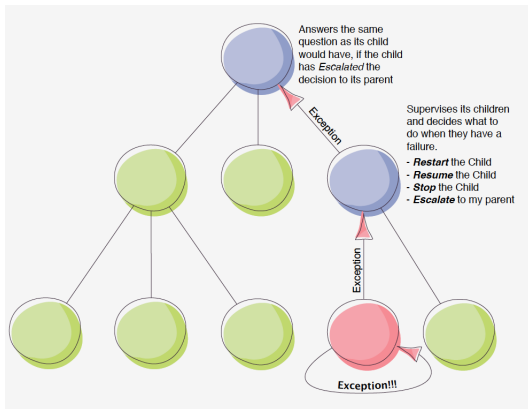
```
throw new Exception("E R R O R!")
```

Tout acteur a un parent donc tout acteur a un superviseur, son parent.

Le parent attrape l'exception et fait un filtrage pour décider ce que l'on doit faire pour traiter le problème.

Supervision récursive

Le superviseur est la partie de l'acteur qui s'occupe de ses enfants.



Stratégies de supervision

```
import akka.actor.{ActorInitializationException,
                  ActorKilledException
                  OneForOneStrategy}
import akka.actor.SupervisorStrategy._

final val supervisorStrategy = OneForOneStrategy() {
  case _: ActorInitializationException => Stop
  case _: ActorKilledException      => Stop
  case _: Exception                 => Restart
  case _                           => Escalate
}
```

- OneForOneStrategy ou AllForOneStrategy
- Decider : le case code

Directives du superviseur

On a 4 possibilités dans la fonction Decider :

- *Stop*, après cela l'acteur (et ses enfants) n'existe plus.
- *Resume*, on fait comme si rien ne s'était passé, on reprend.
- *Escalate*, on ne peut prendre une décision donc on transmet le problème au parent.
- *Restart*, on essaie de reconstituer une nouvelle instance de l'acteur et de re-traiter le message. C'est l'option idéale mais la plus difficile à implémenter.

Le problème est de savoir si on fait de restart ou non sur les enfants. Pas de réponse, mais il y a des heuristiques, par exemple on aime restart les enfants feuilles.

Supervision par le *system*

Que faire si on n'a pas un père ?

Par défaut on fait restart, mais on peut changer la stratégie

```
import akka.actor.SupervisorStrategyConfigurator
import akka.actor.SupervisorStrategy._
import akka.actor.OneForOneStrategy
```

```
class UserGuardianStrategyConfigurator
  extends SupervisorStrategyConfigurator{

  def create(): SupervisorStrategy = {
    OneForOneStrategy() {
      case _ => Resume
    }
  }
}
```

On peut changer la variable de meta-classe `guardiansupervisorstrategy` dynamiquement ou à la configuration.

```
akka {
  actor {
    guardiansupervisorstrategy = UserGuardianStrategyConfigurator
  }
}
```

Que devient le message lors d'une exception ?

Tout dépend de la directive choisie :

- *Stop*, il est perdu à jamais.
- *Resume*, aussi.
- *Escalate*, le père décidera.
- *Restart*, on essaie de prendre une décision à l'aide de hooks `preRestart` et `postRestart`.

preRestart et postRestart

Définition par défaut

```
def preRestart(reason: Throwable, message: Option[Any]) {  
    context.children foreach context.stop  
    postStop()  
}  
  
def postRestart(reason: Throwable) {  
    preStart()  
}
```

l'appelle de postStop() et preStart() n'est pas automatique

Et si on ne peut pas faire Restart

```
override val supervisorStrategy = OneForOneStrategy() {  
    case _: NoDatabaseConnectionException => Restart  
    case _ => Escalate  
}
```

On peut contrôler le nombre de restarts avec :

```
override val supervisorStrategy =  
    OneForOneStrategy(5, 1.minute)
```

qui dit qu'on peut faire maximum 5 restarts par minute.

- Mais comme savoir si un acteur est finalement mort ?
- Comment gérer les acteurs distribués ?